Day 2: Kiro IDE — Custom Agents, Hooks, MCP & Advanced Orchestration

From AI Assistant to Autonomous SDLC Pipeline

Layer 1

Governance

Layer 2

Spec

Layer 3

Custom Agents

Layer 4

Hooks

Layer 5

Steering Files

SECTION 1 — OPENING & RECAP

What You Built Yesterday

Day 1 established the foundation — three critical layers of the governance model. Today you complete the picture by activating Layers 3 and 4.

Layer 1 — Governance

- IAM policy with least-privilege permissions
- Branch protection rules on `main`
- Team governance document committed to the repo

Layer 2 — Spec

- Inventory Service spec — 8 EARS user stories
- Architecture document with DB schema and API design
- Dependency-ordered task plan

Layer 5 — Steering Files

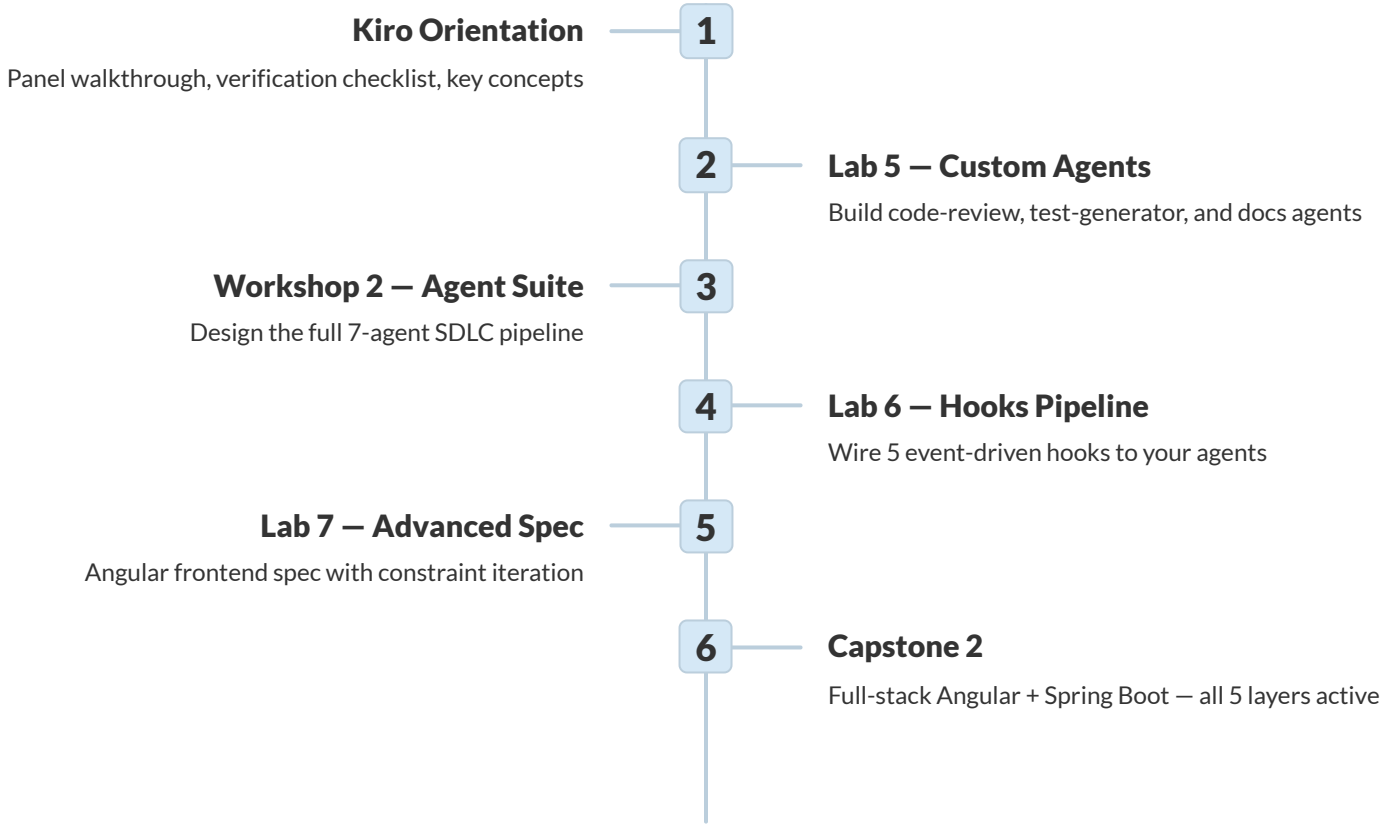
- `architecture.md` — system design standards
- `testing-standards.md` — JUnit 5 + Testcontainers
- `api-design.md` — REST conventions

Today you complete the model — Layers 3 and 4 go live.

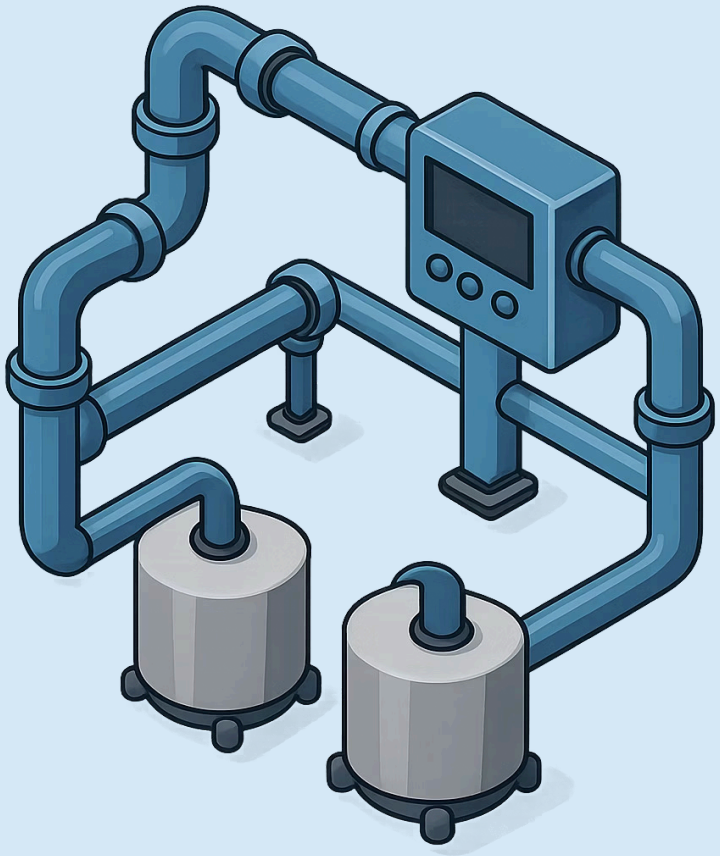
SECTION 1 — OPENING & RECAP

Day 2 Overview

Eight focused hours moving from manual agent invocation to a fully automated, event-driven SDLC pipeline. Every module builds on the last.



All 5 layers active simultaneously by the end of Capstone 2.



SECTION 1 — OPENING & RECAP

The Shift: From On-Demand to Autonomous

Day 1 — You Invoke the Agent

- You type a prompt in the chat panel
- The agent responds to your request
- You review the output and decide what to keep
- The agent is dormant until you speak to it again

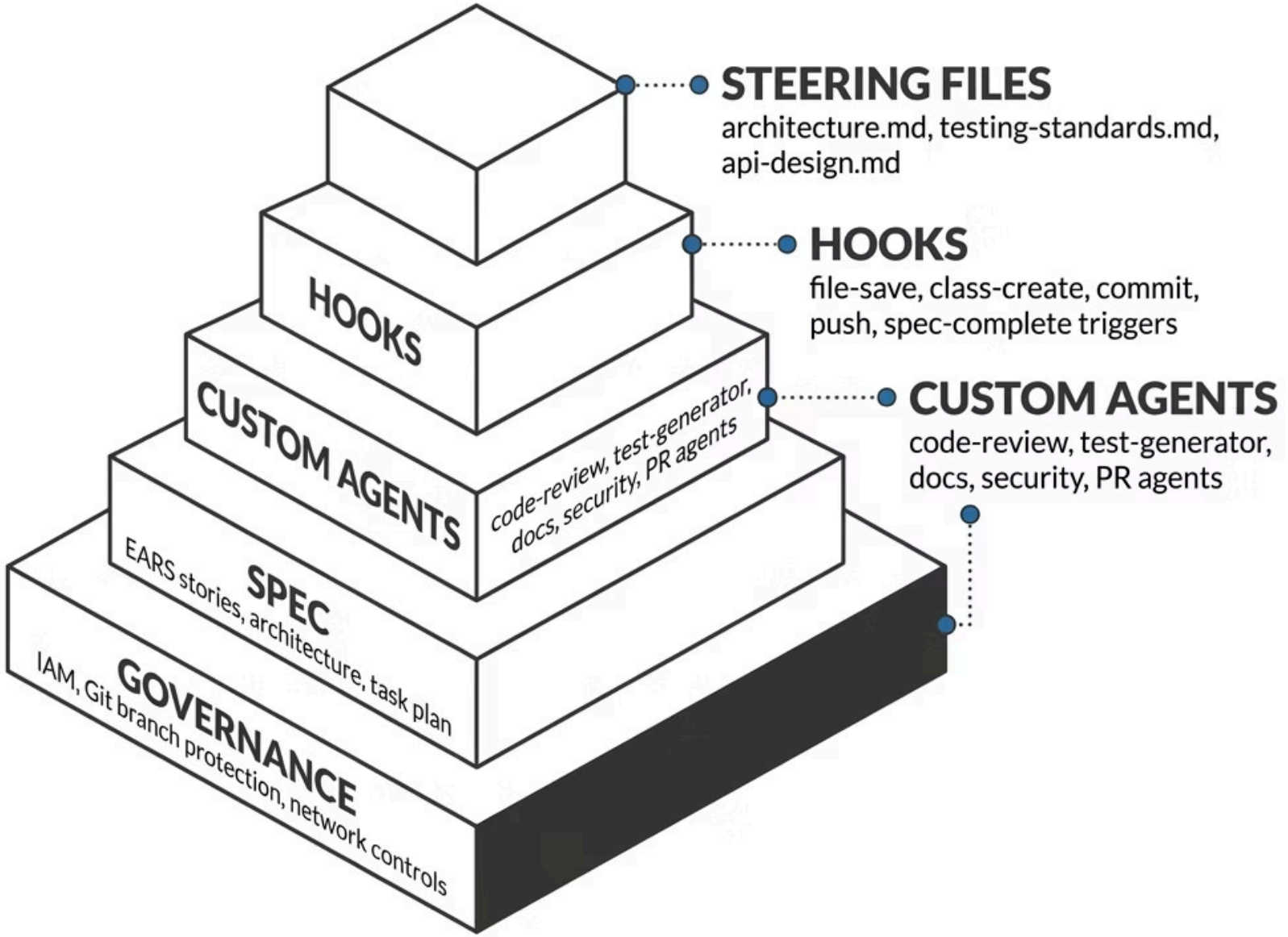
Day 2 — The Pipeline Invokes the Agent

- You save a file — a hook fires automatically
- The agent acts on the event without a prompt
- You review the diff the agent produced
- The agent is a permanent participant, not a tool you pick up

> "The goal is not to use AI faster — it is to make AI a permanent part of the pipeline."

SECTION 1 — OPENING & RECAP

The 5-Layer Governance Model



Day 1 Coverage: Layers 1, 2, and 5 — governance, spec authoring, and persistent context.

Day 2 Coverage: Layers 3 and 4 — custom agents and the event-driven hooks pipeline.

SECTION 1 — OPENING & RECAP

Tools Comparison: Amazon Q Developer vs Kiro IDE

Feature	Amazon Q Developer	Kiro IDE
Distribution	IDE extension (VS Code, IntelliJ)	Standalone IDE built on VS Code
Agentic Mode	Toggle button in the chat panel	Native — always-on agent architecture
Rules / Steering	`.amazonq/rules/`	`.kiro/steering/`
Custom Agents	Not available	`.kiro/agents/` — Markdown-defined
Hooks	Not available	`.kiro/hooks/` — event-driven
MCP Support	Limited	Full MCP server configuration
Best For	Day 1 Java/Spring Boot exercises	Day 2 advanced pipeline features

Same AWS credentials work for both tools — no new account setup required.

MODULE 2.1 — KIRO IDE ORIENTATION

Module 2.1 — Kiro IDE Orientation

Kiro IDE brings spec authoring, custom agents, hooks, and MCP into a single purpose-built environment. If you know VS Code, you already know 90% of Kiro.

VS Code Foundation

All extensions, keybindings, and themes transfer

Spec-Driven by Design

Requirements, design, and tasks are first-class IDE concepts

Agent-Native

Custom agents, hooks, and MCP are built in — not bolt-ons

MODULE 2.1 — KIRO IDE ORIENTATION

What Is Kiro IDE?

Built on VS Code

Every VS Code extension, keybinding, and setting transfers directly. Your team's existing editor configuration works without modification. Kiro adds layers on top — it does not replace the foundation your developers know.

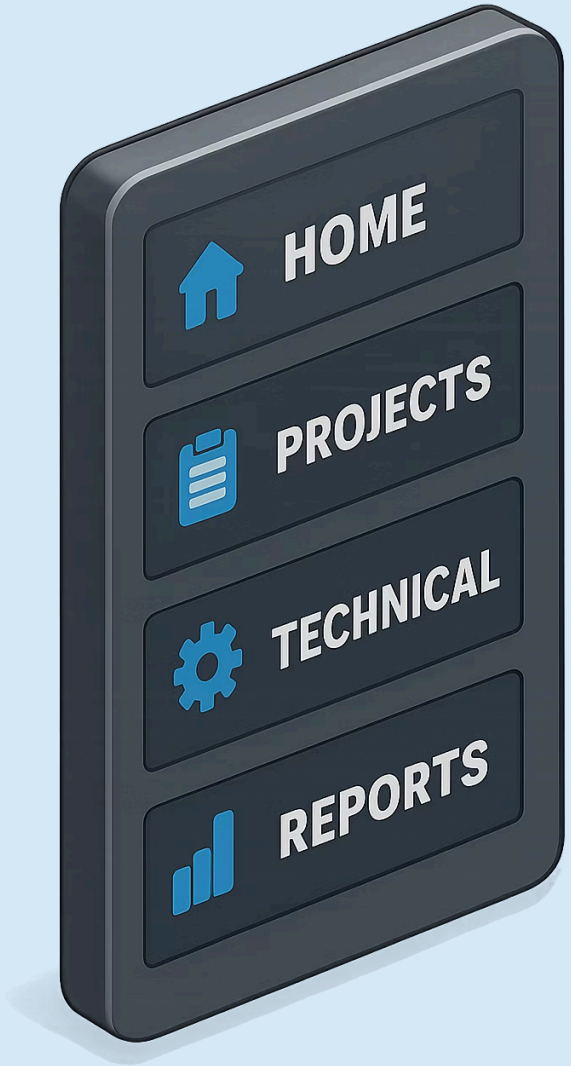
Spec-Driven by Design

Requirements, design documents, and task plans are first-class IDE concepts — not separate Word documents or Confluence pages. The spec panel lives alongside your code, and the agent reads it as authoritative context.

Agent-Native Architecture

Custom agents, hooks, steering files, and MCP server connections are built into the IDE. There is no plugin to install and no configuration bridge — the entire agentic infrastructure is a first-class citizen.

Install from **kiro.dev**. Sign in with your existing AWS credentials.



MODULE 2.1 — KIRO IDE ORIENTATION

The Kiro Panel — Your Control Center

Steering

Displays all `.md` files loaded from `.kiro/steering/`. Shows active context the agent reads before every interaction. Your Day 1 files appear here automatically.

Agent Hooks

Lists all configured hooks with their trigger type and enabled/disabled status. One-click to toggle a hook on or off without editing the JSON file.

Spec

Shows the active spec folder structure — requirements, design, and tasks files for the current feature. Approval status is visible at a glance.

Agents

Directory of all custom agent `.md` files in `.kiro/agents/`. Select any agent to invoke it directly, bypassing Kiro's auto-selection.

MODULE 2.1 — KIRO IDE ORIENTATION

Verification Checklist — Before Labs Begin

Confirm all four items are working before proceeding to Lab 5. If any check fails, resolve it now — later labs depend on this foundation.

1

Repository Open

Kiro opens your training repository and the Ghost panel icon appears in the left sidebar.

2

Steering Panel Populated

The Steering section shows `architecture.md`, `testing-standards.md`, and `api-design.md`.

3

Context Verification

Ask in chat: *"What testing framework for integration tests?"* — the agent must answer **Testcontainers**.

4

Approved Spec Present

`.kiro/specs/inventory-service/` shows the approved spec from Lab 3 with all three stage files.

⚠ If steering files are missing, run the import steps in the lab guide before starting Lab 5.

MODULE 2.1 — KIRO IDE ORIENTATION

Key Kiro Concepts in 90 Seconds

Steering Files

Persistent context the agent reads before every interaction. Stored in `.kiro/steering/`. Your architectural rules and coding standards live here — the agent never forgets them.

Custom Agents

Markdown files in `.kiro/agents/` that define a purpose-built AI with a specific role, a constrained toolset, and a persistent system prompt tailored to one job.

Hooks

Event triggers in `.kiro/hooks/` that fire agents automatically when things happen — file save, git commit, new class creation, or spec task completion.

MCP

Model Context Protocol connects agents to external tools — GitHub, Jira, AWS APIs. An agent with MCP knows your code AND your broader system context.

MODULE 2.1 — KIRO IDE ORIENTATION

Kiro Free vs Pro

Feature	Free Tier	Pro ($20/month)
Agentic Interactions	50 / month	Unlimited
Steering Files	✅ Included	✅ Included
Basic Spec Workflow	✅ Included	✅ Included
Full Hook Pipeline	❌ Not available	✅ Full access
MCP Server Connections	❌ Not available	✅ Full access
Custom Agent Suite	Limited	✅ Unlimited agents

Day 2 labs require Pro tier OR an existing Amazon Q Developer Pro subscription via IAM Identity Center — both provide full access to all features covered today.

LAYER 3 — CUSTOM AGENTS

Layer 3 — Custom Agents

Lab 5 introduces the most powerful layer of the model: purpose-built AI agents with constrained roles, persistent instructions, and version-controlled definitions.

code-review-agent

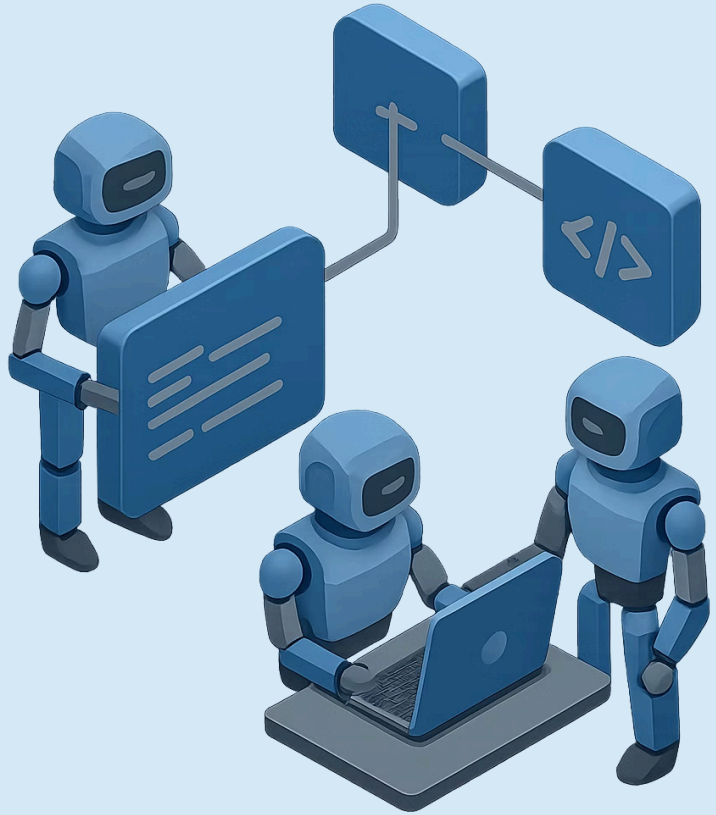
Reads Java code, flags standards violations

test-generator-agent

Generates JUnit 5 tests following AAA pattern

docs-agent

Generates Javadoc and README updates



LAYER 3 — CUSTOM AGENTS

What Is a Custom Agent?

A custom agent is a Markdown file that defines an AI with a specific job, a constrained toolset, and a persistent system prompt. It is the difference between a general-purpose assistant and a specialist who knows your codebase deeply.

name

The agent's identity — used to invoke it directly and displayed in the Kiro panel. Example: `code-review-agent`

description

Determines when Kiro auto-selects this agent based on the developer's prompt. Write it as the sentence a developer would type.

tools

What the agent can do — `read-file`, `create-file`, `execute-command`. Constrain to the minimum required for the job.

system prompt

Persistent instructions, standards, and constraints the agent applies in every interaction — your team's rules encoded in the agent.

ⓘ Agent files live in `.kiro/agents/`. They are version-controlled and shared across the entire team when pulled.

LAYER 3 — CUSTOM AGENTS

Why Constrain Agent Tools?

⚠️ Over-Privileged Agent

- Can read, write, delete, and execute
- Unpredictable side effects during a code review
- May modify production code while reviewing it
- Difficult to audit what changed and why

✅ Correctly Constrained Agent

- `code-review-agent` has read-only access
- Cannot write files, cannot execute commands
- Can only observe — never modify
- Audit trail is clean: the agent touched nothing

A reviewer should never need write access. The principle of least privilege applies to agents just as it does to IAM roles.

LAYER 3 — CUSTOM AGENTS

The Three Agents You Build in Lab 5

code-review-agent

Reads Java code and flags violations of steering file standards, anti-patterns, and test coverage gaps. **Tools:** read-only. Triggered when you ask for a review of a file or PR diff.

test-generator-agent

Generates JUnit 5 tests following AAA pattern and naming conventions from `testing-standards.md`. **Tools:** read + write (creates test files in `src/test`).

docs-agent

Generates Javadoc and README updates from source code. Keeps documentation synchronized with the codebase. **Tools:** read + write. Reads public methods, writes Javadoc comments.

LAYER 3 — CUSTOM AGENTS

Code Review Agent — Deep Dive

The system prompt is where your team's standards become enforced policy. Every rule that currently lives in a code review checklist belongs here.

```
name: code-review-agent
description: >
  Reviews Java code changes for standards compliance,
  anti-patterns, and testing coverage gaps.
tools:
  - read-file
system_prompt: |
  You are a senior Java engineer performing a code review.
  Apply the following rules to every review:

  1. Flag any use of @Autowired on fields.
     Correct approach: constructor injection only.
  2. Flag any method that catches Exception without
     rethrowing or logging at ERROR level.
  3. Flag missing @Valid on controller request bodies.
  4. Flag any entity class missing @Version for
     optimistic locking.
  Return findings as a numbered list with severity:
  CRITICAL, WARNING, or SUGGESTION.
```

This agent applies your steering file standards to every code review — consistently, without fatigue, and without forgetting.

LAYER 3 — CUSTOM AGENTS

Test Generator Agent — Deep Dive

Agent Configuration

```
name: test-generator-agent
description: >
  Generates JUnit 5 tests for a Java class
  following AAA pattern and naming conventions.
tools:
  - read-file
  - create-file
system_prompt: |
  Read the class under test and
  testing-standards.md before generating.

  Naming convention:
  methodName_stateUnderTest_expectedBehaviour

  Structure every test with:
  // Arrange
  // Act
  // Assert

  Use Testcontainers for any test that
  requires a database or external service.
```

Before → After

Before: `InventoryService.java` exists with no test coverage.

After: Agent generates `InventoryServiceTest.java` containing:

- `reserveStock_sufficientStock_reservationCreated`
- `reserveStock_insufficientStock_throwsException`
- `reserveStock_concurrentRequests_optimisticLockRetry`
- `releaseReservation_validId_quantityRestored`

LAYER 3 — CUSTOM AGENTS

Docs Agent — Deep Dive

What It Reads

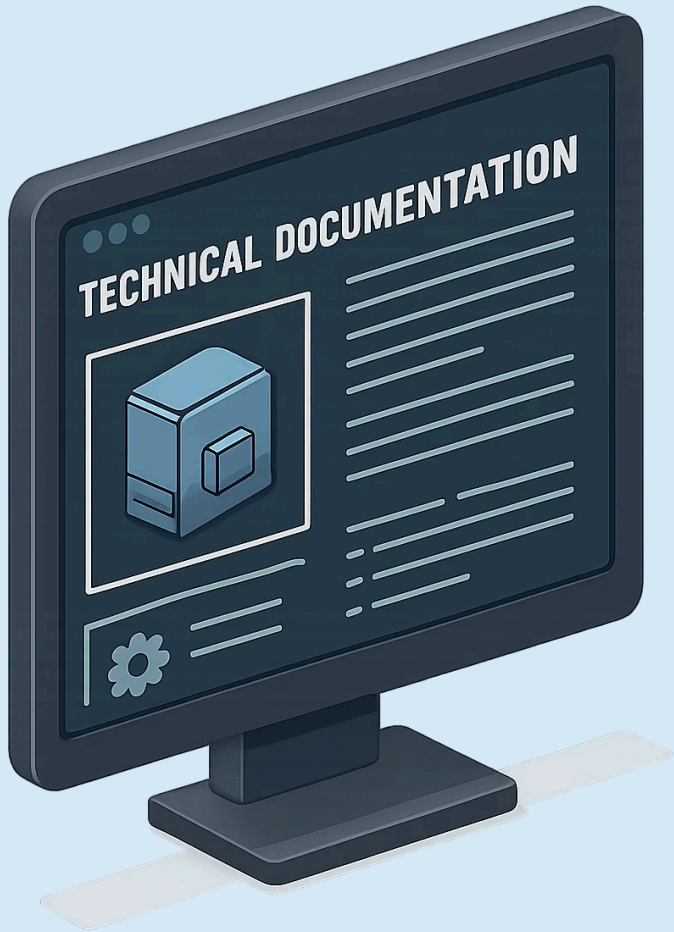
All public methods in the target class — method signature, parameter types, return type, and any existing inline comments. Also reads the current `README.md` endpoint list.

What It Generates

Javadoc blocks with `@param`, `@return`, `@throws`, and `@see` links to related classes. Updates the README endpoint table to reflect the current API surface.

Why It Matters

Documentation that drifts from reality is worse than no documentation — it actively misleads developers. This agent keeps documentation synchronized with every code change.



LAYER 3 — CUSTOM AGENTS

How Kiro Selects an Agent Automatically

The Description Field is the Key

Kiro reads every agent's `description` field and matches it to the semantic meaning of your prompt. The match is semantic, not keyword-based.

Good description:

> "Reviews Java code changes for standards compliance, anti-patterns, and testing coverage gaps."

Bad description:

> "Helps with code."

Auto-Selection in Practice

"Can you review this class?"
→ Kiro selects `code-review-agent`

"Generate tests for UserService"
→ Kiro selects `test-generator-agent`

"Update the Javadoc for this method"
→ Kiro selects `docs-agent`

"What is the API Gateway config?"
→ Kiro uses general context, no specialist selected

Write descriptions that match the exact sentence a developer would type.

LAYER 3 — CUSTOM AGENTS

Agent File Structure — Complete Reference

```
---
name: code-review-agent
description: >
 Reviews Java code changes for standards compliance,
 anti-patterns, and testing coverage gaps.

tools:
 - read-file #
```

LAYER 3 — CUSTOM AGENTS

Lab 5 Deliverable

Three agents built, tested, and committed. Your entire team inherits them on the next `git pull`.

code-review-agent

`.kiro/agents/code-review-agent.md`
Verified: flags `@Autowired` field injection violations on the first review attempt.

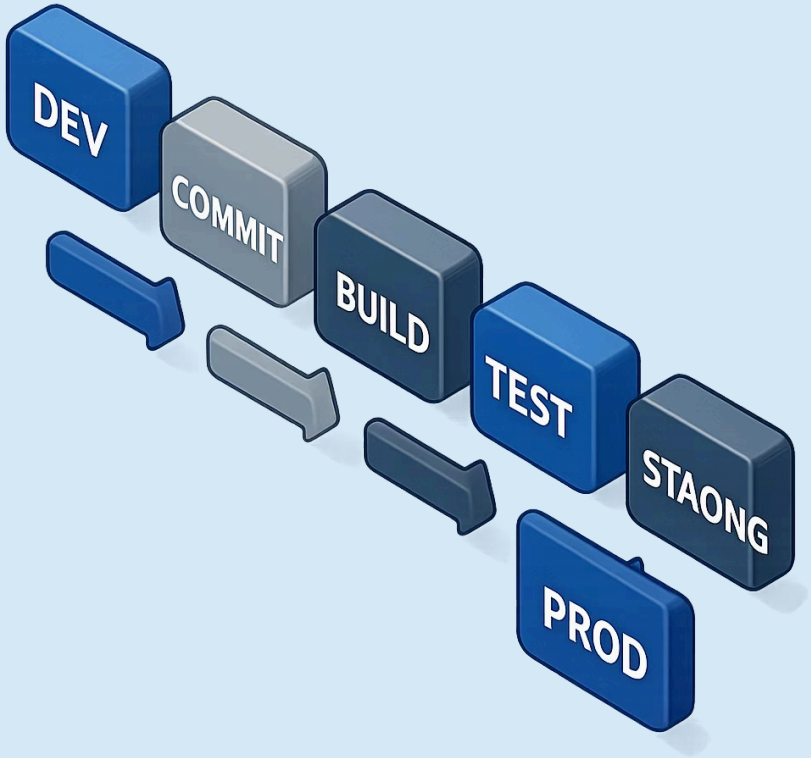
test-generator-agent

`.kiro/agents/test-generator-agent.md`
Verified: generates tests using the `methodName_stateUnderTest_expectedBehaviour` naming convention.

docs-agent

`.kiro/agents/docs-agent.md`
Verified: generates complete Javadoc for `UserService` with all public methods covered.

All three files committed to Git — your whole team inherits these agents automatically.



WORKSHOP 2 — FULL AGENT SUITE

Workshop 2 — Designing Your Full SDLC Agent Suite

Every stage of the software development lifecycle — from feature request to merged PR — can be supported by a purpose-built agent. Workshop 2 designs the complete picture.

WORKSHOP 2 — FULL AGENT SUITE

The 7-Agent SDLC Suite

spec-author-agent

Layer 2 — Generates EARS user stories from a natural-language feature prompt

architect-agent

Layer 2 — Designs DB schema, API endpoints, and data flow diagrams

code-review-agent

Layer 3 — Built in Lab 5 — standards compliance and anti-pattern detection

test-generator-agent

Layer 3 — Built in Lab 5 — JUnit 5 test generation

docs-agent

Layer 3 — Built in Lab 5 — Javadoc and README synchronization

security-agent

Layer 3 — Scans for OWASP issues, hardcoded secrets, and missing validation

pr-agent

Layer 3 — Generates PR descriptions from git diff and spec task references

You built 3 in Lab 5. Workshop 2 designs and adds the final 4.

WORKSHOP 2 — FULL AGENT SUITE

spec-author-agent Design

System Prompt — Key Instructions

You generate EARS user stories using the WHEN/THE SYSTEM SHALL format.

Rules you must never violate:
1. Every happy path story requires a corresponding failure path story.
2. Every feature with shared state requires a concurrency edge case story.
3. Do not proceed to architecture until all stories have been explicitly approved.

Read architecture.md before generating any story that touches persistence.

What This Replaces

In Lab 3, you manually composed the EARS prompt, remembered the concurrency rule, and checked for failure-path coverage by hand.

The `spec-author-agent` encodes all of that institutional knowledge. A junior developer typing *"add a reservation cancellation feature"* gets the same quality of stories as your most experienced architect.

The EARS format is now an enforced standard, not an aspirational practice.

WORKSHOP 2 — FULL AGENT SUITE

architect-agent Design

```
name: architect-agent
description: >
  Designs PostgreSQL schema, REST API endpoints,
  and data flow for a new feature.

tools:
  - read-file
  - create-file

system_prompt: |
  You design features following api-design.md
  and architecture.md conventions.

  Rules:
  1. All entities require a @Version column for
     optimistic locking on concurrent writes.
  2. REST endpoints follow api-design.md — no
     exceptions without explicit approval.
  3. SQS events are published AFTER transaction
     commit — never inside the transaction boundary.
  4. Every schema change requires a Flyway migration
     file with a rollback-safe strategy.

  Return: schema DDL, endpoint table, sequence
  diagram, and Flyway migration script.
```

The correctness rules you applied manually in Lab 3 Stage 2 are now automatically enforced on every new feature.

WORKSHOP 2 — FULL AGENT SUITE

security-agent & pr-agent Design

security-agent

Reads: All Java files in `src/`

Flags:

- Hardcoded credentials or API tokens
- SQL injection risk (string concatenation in queries)
- Missing `@Valid` on controller request bodies
- Use of `java.util.Random` for security purposes
- Disabled CSRF or CORS misconfiguration

Returns: Numbered findings list with severity — CRITICAL, HIGH, MEDIUM

Tools: Read-only — security review never modifies code

pr-agent

Reads: Git diff (via MCP GitHub tool) + `tasks.md` from the active spec

Generates:

- PR title following conventional commit format
- Summary of what changed and why
- Table mapping commits to spec task IDs
- Checklist of what reviewers should verify

Tools: Read-only + MCP GitHub access

Requires: `github-mcp` server configured in `.kiro/mcp.json`

WORKSHOP 2 — FULL AGENT SUITE

Workshop 2 Output

By the end of the workshop, your team has a complete 7-agent SDLC suite committed to version control and available to every developer on pull.

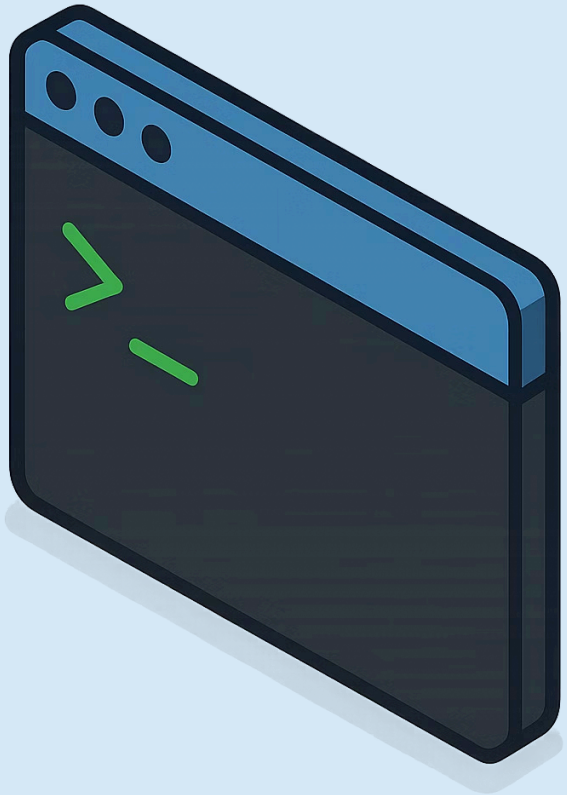
Complete File Tree

```
.kiro/
└── agents/
    ├── spec-author-agent.md
    ├── architect-agent.md
    ├── code-review-agent.md
    ├── test-generator-agent.md
    ├── docs-agent.md
    ├── security-agent.md
    └── pr-agent.md
```

Discussion Prompts

Take 5 minutes to discuss with your lab partner:

- Which agent will save your team the most time in the next sprint?
- Which agent would you build first for your real project?
- Which of your current code review checklist items could live in the security-agent system prompt?



KIRO CLI & MCP

Kiro CLI & MCP — Operations at Scale

The IDE is for developers working interactively. The CLI is for pipelines, automation, and remote execution — where there is no screen and no human in the loop.

CI/CD Pipelines

Run agents in GitHub Actions without opening the IDE

Remote Execution

Fire agents on SSH sessions and cloud instances

Pre-Commit Hooks

Invoke security-agent before every commit from the terminal

Scripted Chains

Chain multiple agent calls in bash for complex automations

KIRO CLI & MCP

Why Kiro CLI?

The same `.kiro/` configuration, the same custom agents, and the same steering files — accessed entirely from the command line, no GUI required.

CI/CD Integration

Run security-agent and code-review-agent inside a GitHub Actions workflow. Any findings above CRITICAL severity can fail the build — automated quality gates without human intervention.

Remote & Headless Operation

Fire agents on EC2 instances, in Docker containers, or inside a Kubernetes job. No display server, no browser, no IDE process required.

Scripted Automation

Chain agents in bash: run test-generator-agent for every new class in a PR, then run code-review-agent on each generated test file, then post a summary to Slack via MCP.

```
kiro-cli chat "Review src/main/java/UserService.java for security issues"
```

KIRO CLI & MCP

Kiro CLI Core Commands

Command	Description
`kiro-cli`	Enter interactive chat session (REPL mode)
`kiro-cli chat "prompt"`	Single non-interactive prompt — returns output and exits
`kiro-cli --no-interactive "prompt"`	CI-safe mode — no confirmation prompts, fully automated
`kiro-cli /login`	Authenticate with AWS credentials
`kiro-cli /logout`	Sign out and clear cached credentials
`kiro-cli /status`	Show authentication and configuration status
`kiro-cli /help`	List all available commands and options

⚠ Always use `--no-interactive` in CI pipelines — agents run fully automated with no human confirmation prompts, making the pipeline non-blocking.

KIRO CLI & MCP

What Is MCP?

Model Context Protocol is an open standard for connecting AI agents to external tools and data sources. Without MCP, your agent only knows what is in the codebase. With MCP, it knows what is in your entire system.

Kiro Agent ↔️ GitHub MCP

The agent can read PR context, list open issues, retrieve commit history, and fetch the full git diff for any branch — without you copying and pasting anything.

Kiro Agent ↔️ Jira MCP

The agent reads sprint tickets for context when generating code, writes completion status back to Jira when spec tasks are done, and links PRs to their originating tickets automatically.

Kiro Agent ↔️ AWS Labs MCP

The agent queries live AWS resources — Lambda function configurations, CloudWatch alarm states, S3 bucket policies, and ECS service health — making it context-aware about the deployed system.

KIRO CLI & MCP

MCP Configuration File

The `.kiro/mcp.json` file declares which external tool servers your agents can connect to. It is workspace-level configuration — committed to Git and reviewed like any other infrastructure change.

```
{
  "mcpServers": {
    "github-mcp": {
      "type": "stdio",
      "command": "github-mcp-server",
      "args": ["--read-only"],
      "env": {
        "GITHUB_TOKEN": "$GITHUB_TOKEN"
      }
    },
    "aws-labs-mcp": {
      "type": "stdio",
      "command": "aws-labs-mcp-server",
      "args": ["--region", "us-east-1"],
      "env": {
        "AWS_ACCESS_KEY_ID": "$AWS_ACCESS_KEY_ID",
        "AWS_SECRET_ACCESS_KEY": "$AWS_SECRET_ACCESS_KEY"
      }
    }
  }
}
```

⊗ Use environment variable references (`$GITHUB_TOKEN`) — never hardcode secrets directly in this file. `mcp.json` will be in your Git history.

KIRO CLI & MCP

MCP Scopes — Workspace vs User

Workspace Scope

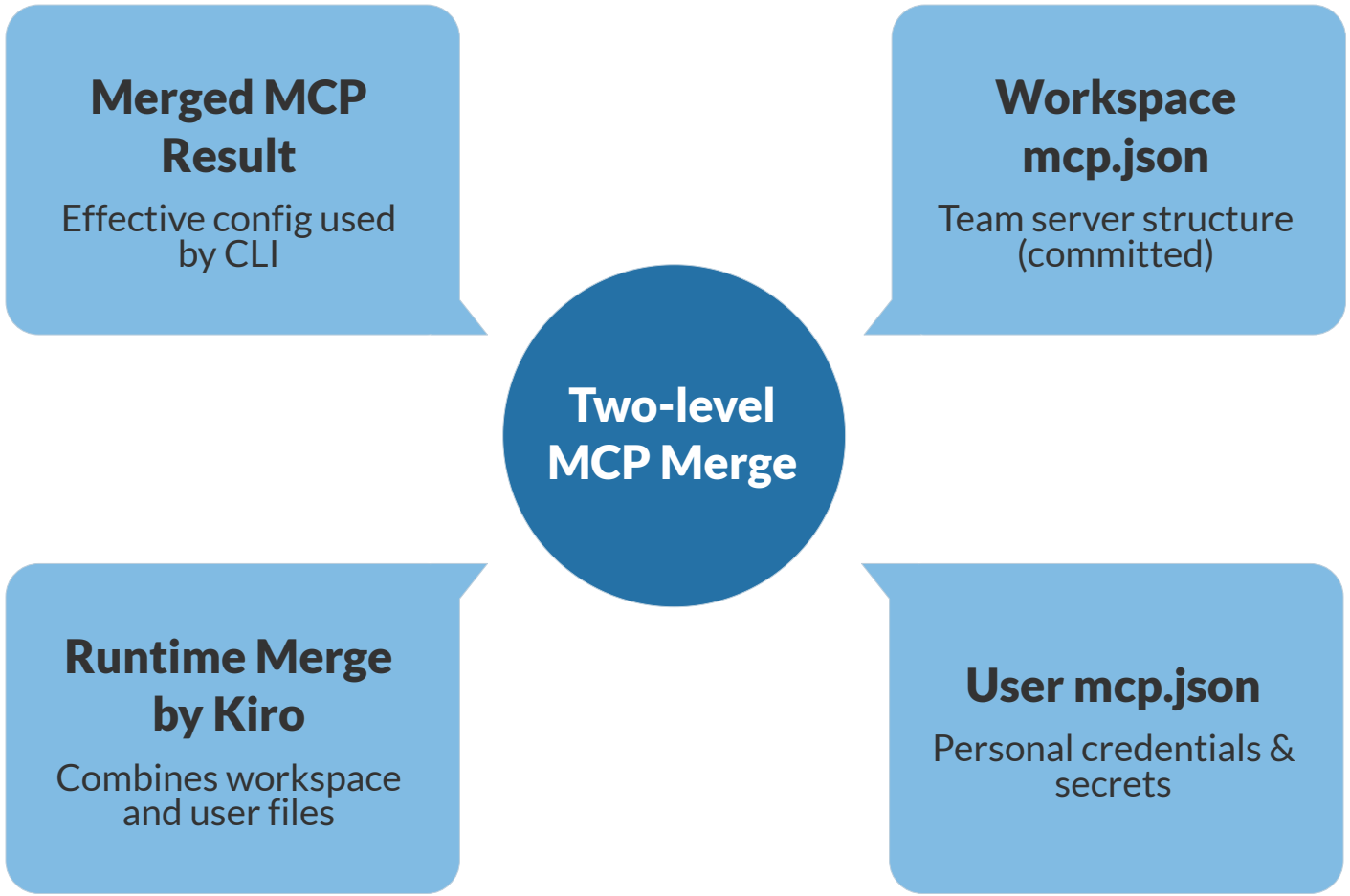
`.kiro/mcp.json` — committed to Git

- Shared across the whole team
- Contains server definitions and non-secret config
- Reviewed in pull requests like any config change
- Defines *which* servers exist and *how* to connect

User Scope

`~/.kiro/mcp.json` — local only, never committed

- Personal credentials and API tokens
- Overrides workspace config for the same server name
- Survives team member onboarding and offboarding
- Defines *who* can authenticate to each server



Define servers at workspace level. Inject secrets at user level. This separation survives team member changes.

KIRO CLI & MCP

Wiring MCP to Custom Agents

pr-agent.md with MCP Declaration

```
name: pr-agent
description: >
  Generates PR descriptions from git diff
  and spec task references.
tools:
  - read-file
mcpServers:
  - github-mcp
system_prompt: |
  1. Use github-mcp get-pr-diff to fetch
     the full diff for the current branch.
  2. Read .kiro/specs/*/tasks.md to find
     which task IDs are addressed.
  3. Generate a PR description with:
     - Summary of changes
     - Task ID to commit mapping table
     - Reviewer checklist
  Never include sensitive data in the PR
  description body.
```

The Execution Chain

1 **Developer Request**

"Generate a PR description"

2 **pr-agent Invoked**

Kiro selects based on description match

3 **MCP Tool Call**

`github-mcp get-pr-diff` fetches the diff

4 **PR Description Generated**

With spec task links and reviewer checklist

KIRO CLI & MCP

MCP Security Rules

Rule 1 — No Secrets in Files

Never put API tokens, passwords, or credentials in `mcp.json`. Use environment variable references: `$GITHUB_TOKEN`. The file will be in Git history — treat it as public.

Rule 2 — Minimum Tool Scope

Grant each MCP server only the tools the agent actually needs. Do not grant full repo admin access when read-only suffices. Scope the permission to the action.

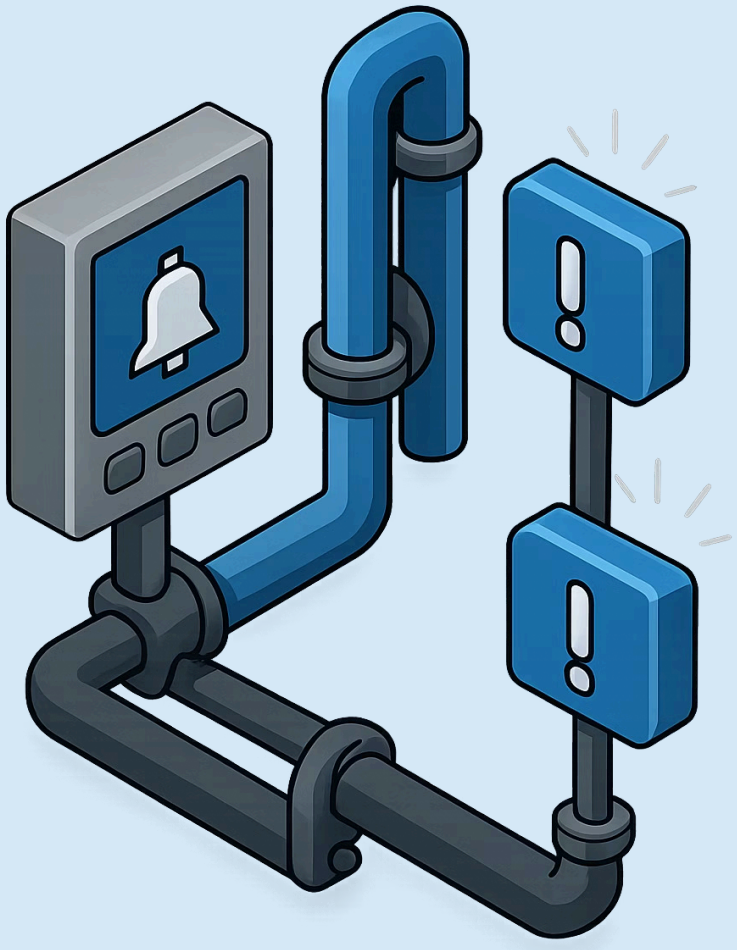
Rule 3 — Review in PR

Every change to `mcp.json` is a new permission grant. Review it in a pull request like an IAM policy change — because that is functionally what it is.

Rule 4 — Separation of Concerns

Workspace scope for server definitions. User scope for credentials. This separation survives team member changes and ensures no personal credentials ever reach version control.

MCP permissions follow the same least-privilege principle as IAM — apply the same rigor.



LAYER 4 — AGENT HOOKS

Layer 4 — Agent Hooks

Lab 6 activates the most transformative layer of the model. Hooks are the mechanism that turns individual agents into a coordinated, event-driven pipeline that runs without being asked.

> "The difference between a developer using AI and a team running AI."

Event-Driven

Agents fire when things happen — not when you remember to ask

Zero Friction

The developer workflow is unchanged — save a file, get a result

Version-Controlled

Hook config lives in `.kiro/hooks/` — shared, reviewed, inherited

LAYER 4 — AGENT HOOKS

What Is a Hook?

A hook is an event binding that fires an agent automatically when a specified event occurs — no prompt required, no developer action needed beyond the work they were already doing.

The Old Model

The agent is a tool you pick up, use, and put down. Documentation happens when someone remembers to ask for it. Tests happen when there is time.

The Hook Model

The agent is a participant that responds to the natural rhythm of development. Documentation happens every time you save. Tests appear when you create a new class.

Hooks turn agents from assistants into pipeline participants — they are always watching, always ready.

LAYER 4 — AGENT HOOKS

Hook Event Types

File Events

Fires when a file matching a glob pattern is saved, created, or deleted. Pattern example: `src/**/*.java`. Use for documentation, test generation, and linting on every save.

Prompt Events

Fires when a specific phrase appears in a chat message. Pattern example: `"ready to commit"`. Turns natural developer language into pipeline triggers — no extra commands to memorize.

Spec Task Events

Fires when a spec task is marked complete in `tasks.md`. Use for end-of-task automations like PR description generation — fires exactly when the work is done.

Manual Trigger

Fires when you click the hook's play button in the Kiro panel. Use for expensive operations you want to control explicitly — full README sync, comprehensive security audit.

LAYER 4 — AGENT HOOKS

The 5-Hook Pipeline You Build in Lab 6

javadoc-on-save

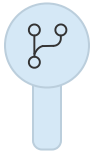
Trigger: `.java` file saved. Agent: `docs-agent`. Action: updates Javadoc for changed methods without overwriting existing documentation.

test-on-new-class

Trigger: new `.java` file created in `src/main`. Agent: `test-generator-agent`. Action: creates a corresponding test skeleton in `src/test`.

security-lint-on-commit

Trigger: prompt event — developer types *"ready to commit"*. Agent: `security-agent`. Action: scans staged files before commit executes.



pr-description-on-push

Trigger: spec task marked complete. Agent: `pr-agent`. Action: generates PR description from git diff + spec task references.

doc-sync-on-stop

Trigger: manual (play button). Agent: `docs-agent`. Action: syncs README with the current endpoint list from all controllers.

LAYER 4 — AGENT HOOKS

Hook File Structure — Complete Reference

```
{
  "name": "javadoc-on-save",
  "description": "Updates Javadoc when a Java source file is saved",
  "enabled": true,
  "event": {
    "type": "file",
    "pattern": "src/**/*.java",
    "trigger": "save"
  },
  "agent": "docs-agent",
  "instructions": "Update Javadoc for all public methods that
    changed in the saved file. Do not modify existing Javadoc
    blocks that are still accurate. If a method has no Javadoc,
    generate a complete block with @param, @return, and @throws.",
  "options": {
    "timeout_seconds": 30,
    "max_retries": 1
  }
}
```

Hook files live in `.kiro/hooks/` — version-controlled, reviewed in PRs, and inherited by the entire team on pull.

LAYER 4 — AGENT HOOKS

Hooks in the Development Workflow

Without Hooks

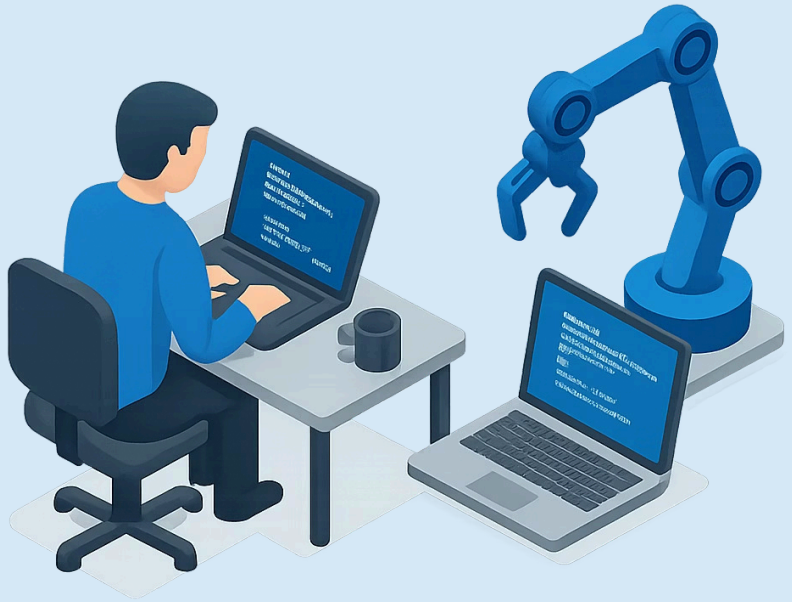
1. Developer writes the code
2. Developer *remembers* to run code review
3. Developer *remembers* to run test generation
4. Developer *remembers* to write Javadoc
5. Developer *crafts* a PR description manually
6. Reviewer still doesn't know which spec tasks are addressed

With Hooks

1. Developer writes the code
2. Developer saves the file
3. All four agents fire automatically
4. Developer reviews the diffs the agents produced

Four steps become two. The cognitive burden of remembering to run quality tools disappears entirely.

The developer's job shifts from running tools to reviewing what the tools produced — a fundamentally better use of expert attention.



LAYER 4 — AGENT HOOKS

Hook Governance — Default Policies

Not every hook should be enabled by default. The `enabled` field in each hook file controls this — make the policy explicit and commit it to the repository.

✅ Always On

`javadoc-on-save`
`test-on-new-class`

Low risk. High value. Output is reversible. No developer action required — runs silently and produces diffs for review.

⚙️ Opt-In Required

`security-lint-on-commit`

May interrupt fast iteration cycles. Developers enable individually when they are ready to commit. Default: `enabled: false`.

🖱️ Manual Only

`doc-sync-on-stop`

Heavy operation — scans all controllers and updates README comprehensively. Run when genuinely needed, not on every save. Default: `enabled: false`.

Default to `false` for expensive hooks. Let developers opt in rather than opt out.

LAYER 4 — AGENT HOOKS

Lab 6 Deliverable

Five hooks configured, tested, and committed. Verify each one by triggering the corresponding event manually before marking the lab complete.

javadoc-on-save

Fires and updates Javadoc when you save `UserService.java`

test-on-new-class

Creates a test skeleton when you create `InventoryController.java`

security-lint-on-commit

Catches the hardcoded password in the lab exercise before the commit executes

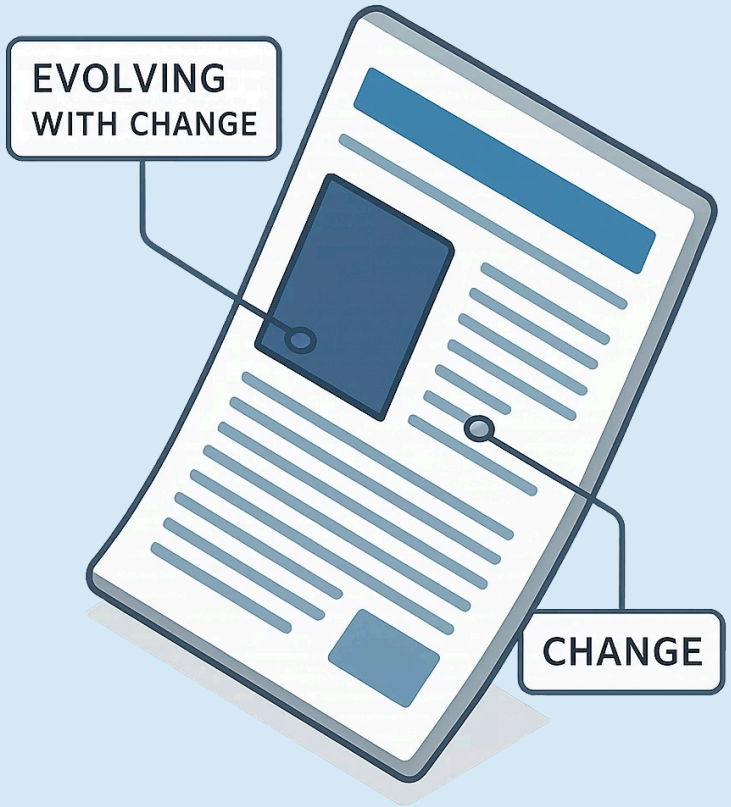
pr-description-on-push

Generates a PR description from the Capstone 1 diff with spec task links

doc-sync-on-stop

Updates the README endpoint table when triggered manually

All 5 hooks committed to `.kiro/hooks/` — test them by triggering each event manually before moving to Lab 7.



LAB 7 — ADVANCED SPEC

Lab 7 — Advanced Spec Iteration

Real specifications change. Features get new constraints, security requirements emerge mid-sprint, and integration dependencies shift. Lab 7 teaches you how to evolve a spec without losing what has already been approved.

> "The most valuable skill in spec-driven development is not writing a good spec — it is changing the spec without starting over."

New Deliverable

Angular 17+ frontend spec for the Inventory Management Service

Integration Contract

Backend spec from Lab 3 defines the API surface — frontend must match

Mid-Spec Change

Security constraint added after Stage 2 is approved — handled without restart

LAB 7 — ADVANCED SPEC

The Angular Frontend Spec

Lab 7 builds a complete spec for the Angular 17+ frontend that consumes the Inventory Management Service API you specced in Lab 3. Enter this feature prompt to begin:

> Build an Angular 17 frontend for the Inventory Management Service. It must show a real-time stock table, allow warehouse staff to submit reservations, and show a reservation history list.
>
> Include an offline mode that queues reservations locally when the API is unreachable and automatically replays them when connectivity is restored.

ⓘ The backend spec from Lab 3 is the integration contract — the frontend spec must match its endpoints exactly. The agent reads both specs before generating architecture.

Components

StockTable, ReservationForm, ReservationList, Dashboard

Offline Mode

IndexedDB queue + connectivity detection + automatic replay

Integration

Consumes `GET /api/v1/stock` and `POST /api/v1/stock/reservations`

LAB 7 — ADVANCED SPEC

EARS Stories for the Angular Frontend

US-01 — Stock Dashboard

WHEN the dashboard loads, THE SYSTEM SHALL fetch current stock levels from `GET /api/v1/stock` and display them in a sortable table with product ID, total quantity, reserved quantity, and available quantity.

US-02 — Reservation Submission

WHEN a warehouse staff member submits a reservation form, THE SYSTEM SHALL `POST` to `/api/v1/stock/reservations` and display the reservation ID on success within 500ms.

US-03 — Offline Queue

IF the API is unreachable when a reservation is submitted, THE SYSTEM SHALL queue the reservation in IndexedDB and display a pending status indicator to the user.

US-04 — Automatic Replay

WHEN connectivity is restored, THE SYSTEM SHALL automatically replay all queued reservations in submission order and update each reservation's status from pending to confirmed or failed.

⚠ US-03 and US-04 are the offline resilience constraint — the agent will not generate these unless you explicitly include the offline requirement in the feature prompt.

LAB 7 — ADVANCED SPEC

Mid-Spec Constraint Change

Stages 1 and 2 have been approved. The security team has now mandated Cognito JWT authentication on all API calls. Here is how to handle it without restarting the spec.

The Change Prompt

Add a cross-cutting constraint to the spec:
all API calls must include a Cognito JWT
Bearer token.

The Angular app must retrieve the token from
AWS Amplify and attach it to every HTTP
request via an interceptor.

Update the architecture and add the
authentication tasks to the task plan at
the correct position in the dependency
order.

Do not modify previously approved user
stories — extend the spec with an addendum.

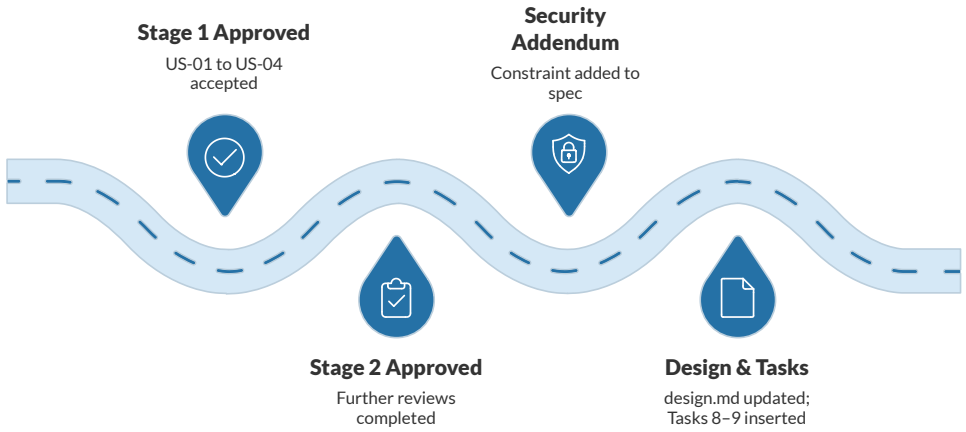
What the Agent Does

- Adds an addendum section to `requirements.md` — previously approved stories are preserved intact
- Updates `design.md` to show API Gateway + Cognito + Amplify interceptor
- Inserts Task 8 (configure AWS Amplify) and Task 9 (implement JWT interceptor) at the correct dependency position
- Flags any existing tasks that now have a new prerequisite

The agent iterates the approved spec rather than starting from scratch — previously approved stories are fully preserved.

LAB 7 — ADVANCED SPEC

Spec Versioning — Change Without Losing History



Before the Constraint Change

design.md shows direct HTTP calls from Angular to the Spring Boot API. US-01 through US-04 approved.

After the Constraint Change

design.md shows Angular → Amplify Interceptor → API Gateway → Cognito → Spring Boot. US-01 through US-04 remain approved and unchanged. New addendum section added.

Approved stories are never deleted — they are extended or superseded with a clear audit trail. The spec is a living document with a complete history.

LAB 7 — ADVANCED SPEC

Lab 7 Deliverable

A complete, versioned Angular frontend spec that reflects a real mid-sprint constraint change — ready to hand directly to Capstone 2 as the agent's task plan.

1

Frontend Spec Approved

Angular frontend spec committed with 6 or more EARS user stories — all in WHEN/THE SYSTEM SHALL format.

2

Offline Resilience Present

US-03 (offline queue) and US-04 (automatic replay) are present and approved in `requirements.md`.

3

Cognito Constraint Added

`design.md` shows API Gateway + Cognito + Amplify interceptor without losing any previously approved stories.

4

Task Plan Updated

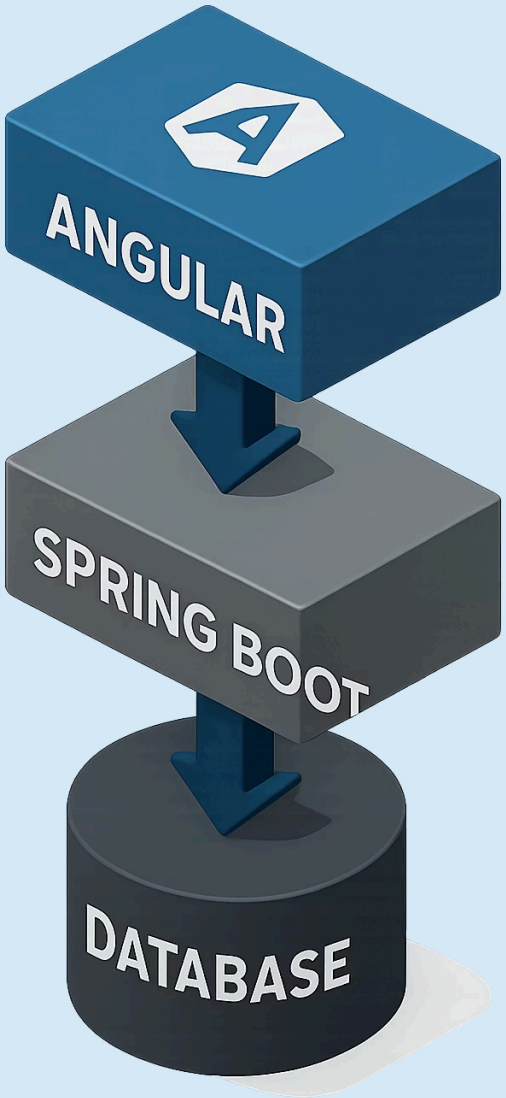
Tasks 8 and 9 (Amplify + JWT interceptor) correctly sequenced with all dependency relationships marked.

5

Spec Committed

`.kiro/specs/angular-frontend/` contains `requirements.md`, `design.md`, and `tasks.md`.

This spec is the direct input for Capstone 2 — the agent will follow this task plan to build the Angular application.



CAPSTONE 2 — ALL 5 LAYERS ACTIVE

Capstone 2 — Full-Stack Angular + Spring Boot

For the first time in this training, all five layers of the governance model are active simultaneously. This is the target state for your real projects.

All 5 layers active simultaneously — the complete governance model in production-pattern operation.

Angular 17 Frontend

StockTable, ReservationForm, ReservationList, Dashboard with offline mode

AWS API Gateway

Cognito JWT authentication layer with Amplify interceptor

Spring Boot API

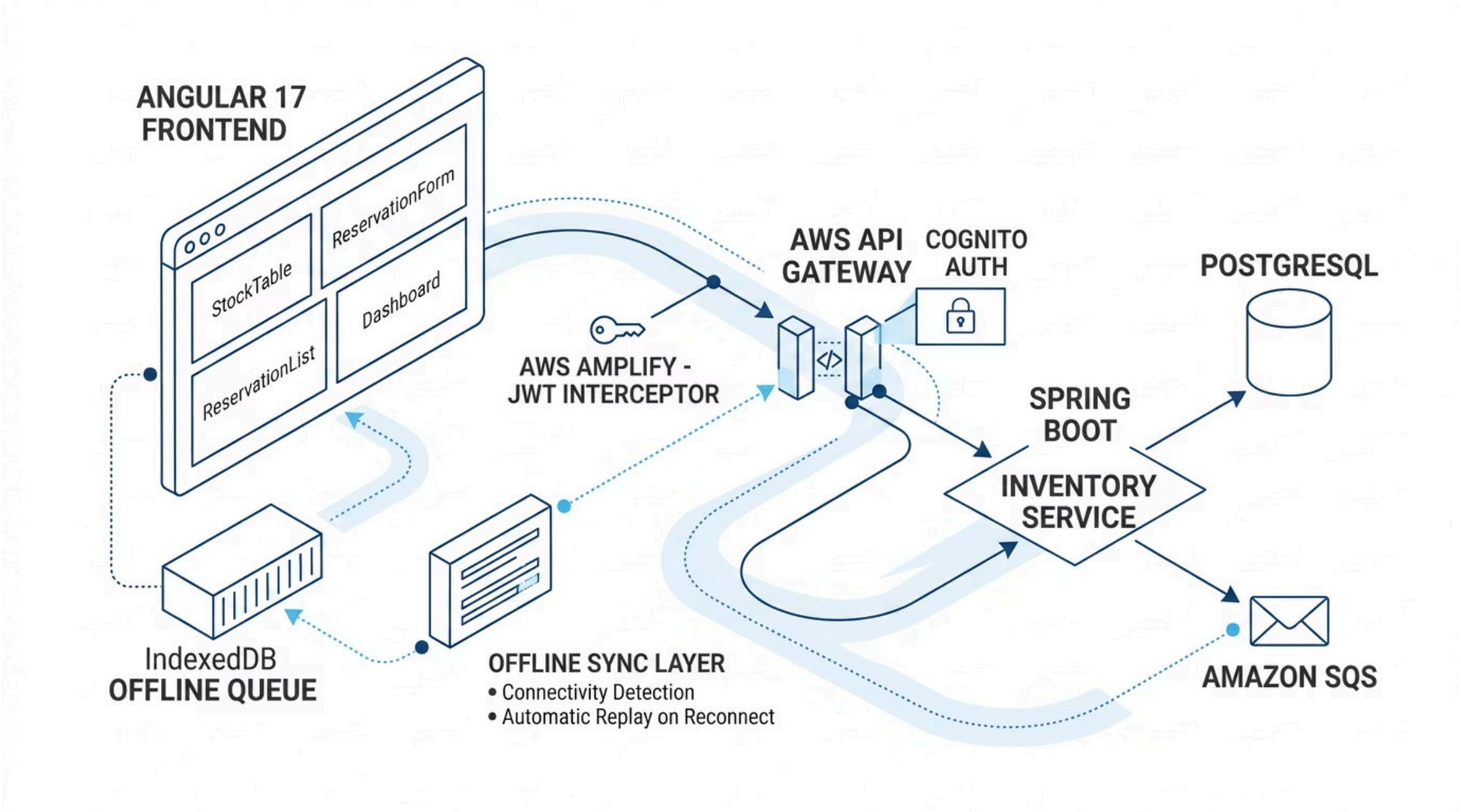
Inventory Service from Capstone 1 — unchanged, fully integrated

PostgreSQL + SQS

Persistence and event messaging as designed in the original spec

CAPSTONE 2 — ALL 5 LAYERS ACTIVE

What You Are Building



The agent implements Tasks 3–6 from your approved Angular spec — component generation, API integration, offline mode, and authentication wiring. Your steering files, hooks, and custom agents are active throughout the entire build.

CAPSTONE 2 — ALL 5 LAYERS ACTIVE

How All 5 Layers Work Together

Layer 5 — Steering Files

`architecture.md` and `api-design.md` inform every architectural decision the agent makes — component structure, service layer patterns, and API call conventions.

Layer 2 — Spec

The Angular frontend spec provides the dependency-ordered task plan. The agent follows Tasks 3–6 in sequence. Each task completion triggers Layer 4.

Layer 3 — Custom Agents

`security-agent` scans generated TypeScript files. `docs-agent` updates the README after each component is generated. `pr-agent` fires when all tasks complete.

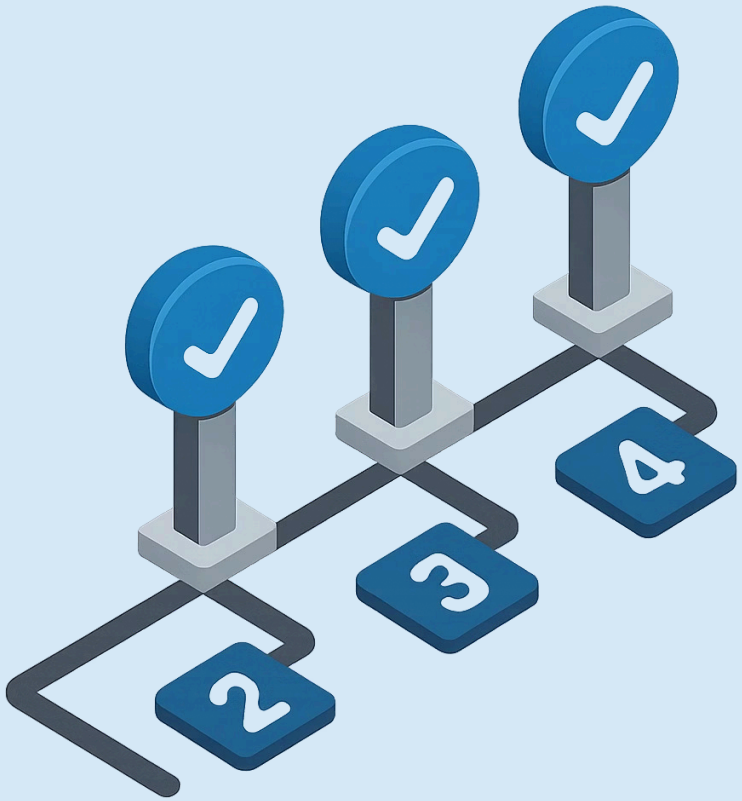
Layer 4 — Hooks

`javadoc-on-save` fires on TypeScript files. `test-on-new-class` creates spec files for new components. `pr-description-on-push` generates the final PR description.

Layer 1 — Governance

IAM policy controls what the agent can do in AWS. Branch protection enforces feature branch workflow — all Capstone 2 output goes to a branch, never directly to `main`.

This is the target state for your real projects — all layers coordinating without manual orchestration.



CAPSTONE 2 — ALL 5 LAYERS ACTIVE

Capstone 2 — Milestones

Milestone 1

Angular components generated and compiling without errors. `StockTableComponent`, `ReservationFormComponent`, and `DashboardComponent` present in the project.

Milestone 2

API integration working. Stock table loads live data from the Spring Boot service. The `GET /api/v1/stock` response renders correctly in the sortable table.

Milestone 3

Offline mode verified. Disconnect network, submit a reservation, queue appears in IndexedDB. Reconnect — automatic replay fires and the reservation confirms.

Milestone 4

Security gate passed. PR agent generates PR description with spec task links. Security agent returns a clean scan. Ready for group demo.

CAPSTONE 2 — ALL 5 LAYERS ACTIVE

The Security Gate

Before merging, all Capstone 2 output must pass a three-check automated security gate. The gate is run by hooks — you review the results, not the findings list.

Check 1 — TypeScript Scan

`security-agent` scans all Angular TypeScript files. Flags: hardcoded API URLs in source, tokens or credentials in any `.ts` file, `console.log` statements containing sensitive data.

Check 2 — Spring Boot Scan

`security-agent` scans all changes to the Spring Boot service. Confirms: no new hardcoded secrets, `@Valid` present on all new controller request body parameters.

Check 3 — PR Traceability

`pr-agent` generates a PR description that explicitly links each commit to a spec task ID. The reviewer can see exactly which approved task each change addresses — no guesswork.

⊗ The gate is automated — hooks fire it, you review the result. A CRITICAL finding from the security agent blocks the merge until resolved.

CAPSTONE 2 — ALL 5 LAYERS ACTIVE

Capstone 2 Deliverable

A production-pattern full-stack application built agent-first with all 5 governance layers active. Ready to demo and ready to use as a reference implementation for your real projects.

1

Stock Dashboard Live

Angular dashboard loads real stock data from the Spring Boot API with correct authentication flow.

2

Reservation Form Works

Reservation form submits successfully and displays the reservation ID returned by the API.

3

Offline Mode Verified

Offline queue stores reservation in IndexedDB and replays it automatically on reconnect.

4

Security Gate Passed

Security agent returns a clean scan with no CRITICAL or HIGH findings.

5

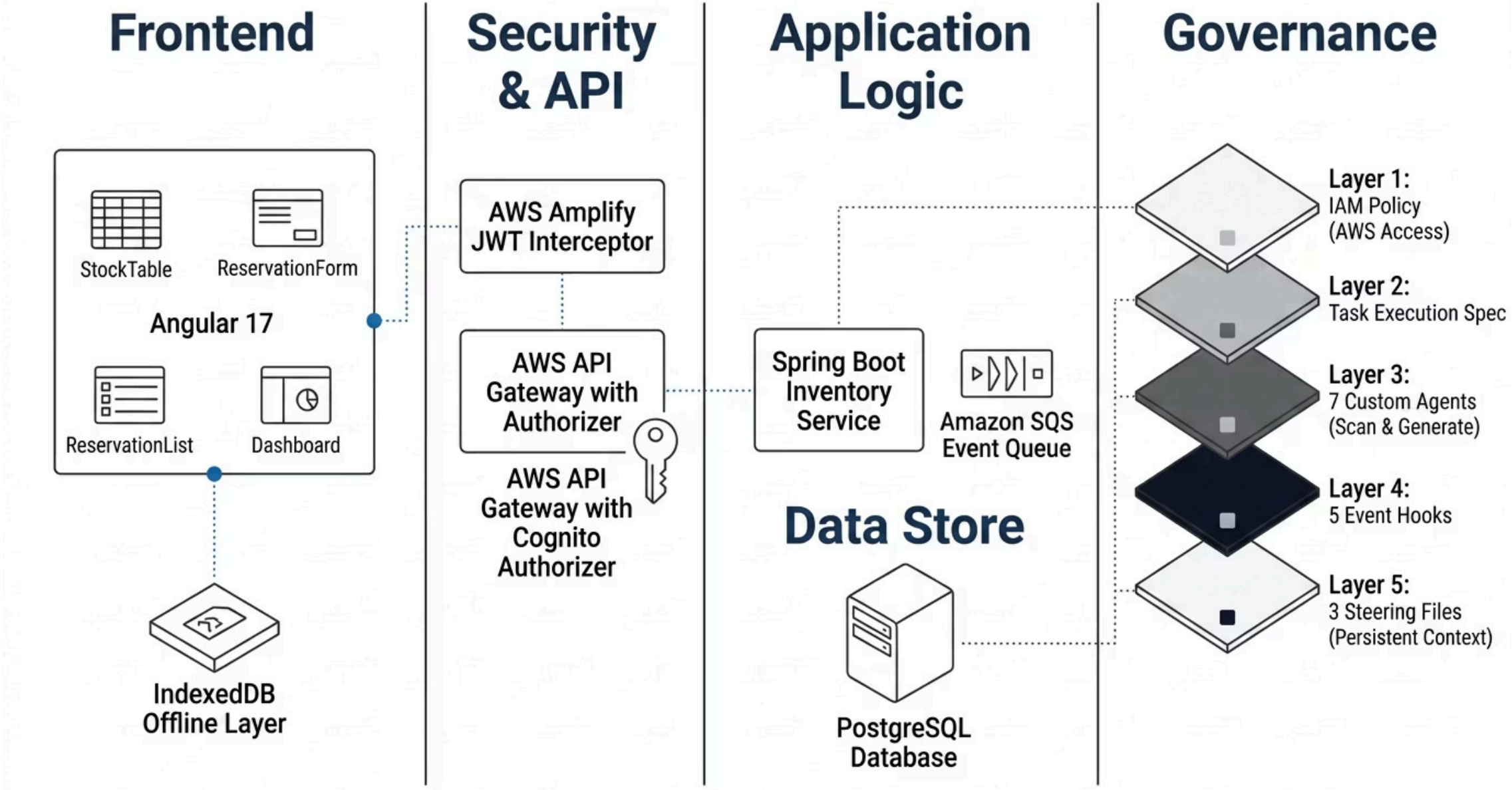
PR Description Generated

PR agent generates a description with spec task references — committed to a feature branch, not `main`.

This is a production-pattern full-stack application built agent-first — the governance model working as designed.

CAPSTONE 2 — ALL 5 LAYERS ACTIVE

The Full-Stack Architecture — Final State



WRAP-UP

The 5-Layer Model — Complete

Layer 1 — Governance ✅

IAM policy, Git branch protection, network tier, governance document

Layer 2 — Spec ✅

EARS stories, architecture design, dependency-ordered task plan

Layer 3 — Custom Agents ✅

7-agent SDLC suite committed to `.kiro/agents/`

Layer 4 — Hooks ✅

5-hook pipeline committed to `.kiro/hooks/`

Layer 5 — Steering Files ✅

Architecture, testing standards, and API design files — persistent team knowledge

This is not a tool. This is a new way of building software.

WRAP-UP

4-Month Adoption Roadmap

You do not adopt all 5 layers at once. You add one layer per month, measure the impact, and build organizational confidence before the next layer goes live.

Month 1 — Foundation

Governance + steering files on one real team project. Establish IAM policy, branch protection, and the three core steering files. Measure: reduction in code review comments about standards violations.

Month 2 — Spec Workflow

Add spec workflow to one new feature. Require spec approval before any code is written. Measure: reduction in rework caused by misunderstood requirements.

Month 3 — Agent Suite

Deploy the 7-agent suite. Code-review-agent in every PR. Test-generator-agent for all new classes. Measure: test coverage delta and code review turnaround time.

Month 4 — Hooks Pipeline

Activate hooks pipeline — javadoc-on-save and security-lint-on-commit across all Java repositories. Measure: security findings caught pre-merge vs. post-deploy.

Add one layer per month. Measure the impact before activating the next layer — evidence builds organizational confidence.

WRAP-UP

What You Brought In vs What You Are Taking Out

What You Brought Into This Room

- GitHub Copilot or similar AI assistant experience
- Deep Java and Spring Boot expertise
- A team writing AI prompts ad-hoc, inconsistently
- No shared standard for how AI fits into the SDLC

What You Are Taking Out

- A 5-layer governance model — systematic, not ad-hoc
- A 7-agent SDLC suite covering the full development lifecycle
- An event-driven hooks pipeline — AI as a permanent participant
- A spec workflow that makes requirements a machine-readable contract
- The ability to design AI as an engineering discipline

> "The developers who will lead the next decade are not the ones who use AI the most — they are the ones who design it best."